Lecture Review

Inheritance is one of the core features of Object-Oriented Programming that allows a class (called the **derived** or **child** class) to acquire the properties and behaviours of another class (called the **base** or **parent** class). Inheritance promotes code reusability, establishes an *is-a* relationship between classes, and forms the backbone of runtime polymorphism.

Real World Example

- Consider a **Vehicle** as a base class. A **Car** and a **Truck** are both vehicles they inherit common characteristics such as having an engine, wheels, and a fuel type. However, each also has its own specialised attributes (e.g., a Car has a sunroof while a Truck has a cargo capacity). This is the essence of inheritance.

Syntax of Inheritance in C++

```
class BaseClass {
    // base class members
};

class DerivedClass : access_specifier BaseClass {
    // derived class members
};
```

The `access_specifier` controls how the members of the base class are accessible in the derived class. It can be `public`, `protected`, or `private`.

Types of Inheritance in C++

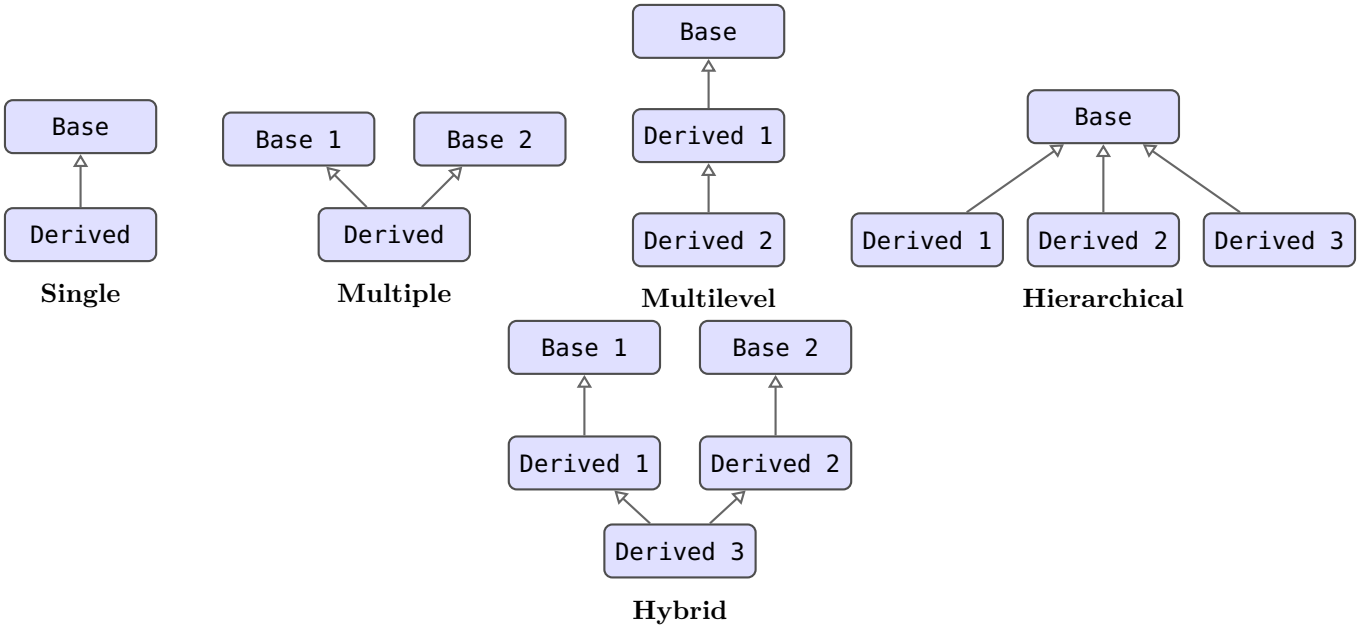

Figure 1: Types of Inheritance in C++

This lab focuses on the **access specifiers** used during inheritance: `public`, `protected`, and `private`.

Access Specifiers: Quick Recap

Before studying how access specifiers affect inheritance, recall what they mean inside a single class:

Specifier	Meaning inside the class
`public`	Accessible from anywhere (inside the class, derived classes, and outside).
`protected`	Accessible inside the class and in derived classes, but *not* from outside.
`private`	Accessible *only* inside the class itself; not accessible in derived classes or from outside.

Table 1: Access Specifiers in C++

Public Inheritance

When a class is derived with `public` inheritance, the access levels of base class members are **preserved** in the derived class:

- `public` members of the base → `public` in the derived class.
- `protected` members of the base → `protected` in the derived class.
- `private` members of the base → **inaccessible** in the derived class (still inherited but hidden).

```cpp
#include <iostream>
using namespace std;

class Animal {
public:
    string name;

    void eat() {
        cout << name << " is eating." << endl;
    }

protected:
    int age;

private:
    string secret; // not accessible in derived class
};

class Dog : public Animal {
public:
    void bark() {
        cout << name << " says: Woof!" << endl; // OK: name is public
        age = 3;                                // OK: age is protected
        // secret = "x";                        // ERROR: private in base
    }
};

int main() {
    Dog d;
    d.name = "Bruno";   // OK: public in derived class
    d.eat();            // OK: inherited as public
    d.bark();
    // d.age = 5;       // ERROR: protected -- not accessible outside
    return 0;
}

Sample output:
Bruno is eating.
Bruno says: Woof!
```

Protected Inheritance

When a class is derived with `protected` inheritance, both `public` and `protected` members of the base class become `protected` in the derived class:

- `public` members of the base → `protected` in the derived class.
- `protected` members of the base → `protected` in the derived class.
- `private` members of the base → **inaccessible** in the derived class.

```cpp
#include <iostream>
using namespace std;

class Vehicle {
public:
    int speed;

    void showSpeed() {
        cout << "Speed: " << speed << " km/h" << endl;
    }

protected:
    string fuelType;
};

class Car : protected Vehicle {
public:
    void setDetails(int s, string f) {
        speed    = s;        // OK: was public, now protected inside Car
        fuelType = f;        // OK: was protected, still protected inside Car
    }
    void display() {
        cout << "Fuel: " << fuelType << endl;
        showSpeed();         // OK: accessible inside derived class
    }
};

int main() {
    Car c;
    c.setDetails(120, "Petrol");
    c.display();

    // c.speed = 200;       // ERROR: speed is protected in Car
    // c.showSpeed();       // ERROR: showSpeed is protected in Car
    return 0;
}

Sample output:
Fuel: Petrol
Speed: 120 km/h
```

Private Inheritance

When a class is derived with `private` inheritance (the default in C++ if no specifier is given), **all** inherited members become `private` in the derived class:

- `public` members of the base → `private` in the derived class.
- `protected` members of the base → `private` in the derived class.
- `private` members of the base → **inaccessible** in the derived class.

```cpp
#include <iostream>
using namespace std;

class Engine {
public:
    int horsepower;

    void start() {
        cout << "Engine started with " << horsepower << " HP." << endl;
    }

protected:
    string engineType;
};

class Truck : private Engine {
public:
    void configure(int hp, string type) {
        horsepower = hp;      // OK: inside Truck -- became private
        engineType = type;    // OK: inside Truck -- became private
    }
    void showInfo() {
        start();              // OK: accessible inside Truck
        cout << "Type: " << engineType << endl;
    }
};

class HeavyTruck : public Truck {
public:
    void test() {
        // horsepower = 500; // ERROR: private in Truck -- not accessible here
        showInfo();           // OK: showInfo() is public in Truck
    }
};

int main() {
    Truck t;
    t.configure(400, "Diesel V8");
    t.showInfo();

    // t.horsepower = 500;  // ERROR: private in Truck
    // t.start();           // ERROR: private in Truck

    HeavyTruck ht;
    ht.configure(600, "Turbo Diesel");
    ht.test();
    return 0;
}

Sample output:
Engine started with 400 HP.
Type: Diesel V8
```

```
Engine started with 600 HP.
Type: Turbo Diesel
```

Summary Table: Inherited Access Levels

Base Class Member	Public Inheritance	Protected Inheritance	Private Inheritance
public	public	protected	private
protected	protected	protected	private
private	Inaccessible	Inaccessible	Inaccessible

Table 2: Effect of Inheritance Type on Member Access in the Derived Class

Multilevel and Hierarchical Inheritance Example

```cpp
#include <iostream>
using namespace std;

// Base class
class Person {
public:
    string name;
    int    age;

    void displayBasic() {
        cout << "Name: " << name << "  Age: " << age << endl;
    }
};

// Derived from Person (public) -- Hierarchical
class Student : public Person {
public:
    int rollNo;

    void displayStudent() {
        displayBasic();
        cout << "Roll No: " << rollNo << endl;
    }
};

// Derived from Student (public) -- Multilevel
class GradStudent : public Student {
public:
    string researchTopic;

    void displayGrad() {
        displayStudent();
        cout << "Research: " << researchTopic << endl;
    }
};
```

```
int main() {
    GradStudent gs;
    gs.name          = "Ayesha";
    gs.age           = 24;
    gs.rollNo        = 2021345;
    gs.researchTopic = "Deep Learning";
    gs.displayGrad();
    return 0;
}

Sample output:
Name: Ayesha  Age: 24
Roll No: 2021345
Research: Deep Learning
```

Constructor and Destructor Behaviour in Inheritance

When a derived class object is created, **the base class constructor is called first**, followed by the derived class constructor. When the object is destroyed, the **derived class destructor is called first**, followed by the base class destructor.

```
#include <iostream>
using namespace std;

class Base {
public:
    Base()  { cout << "Base Constructor called."  << endl; }
    ~Base() { cout << "Base Destructor called."   << endl; }
};

class Derived : public Base {
public:
    Derived()  { cout << "Derived Constructor called."  << endl; }
    ~Derived() { cout << "Derived Destructor called."   << endl; }
};

int main() {
    Derived obj;
    return 0;
}

Sample output:
Base Constructor called.
Derived Constructor called.
Derived Destructor called.
Base Destructor called.
```

To pass arguments to the base class constructor, use the *initialiser list*:

```
class Employee : public Person {
private:
```

```
    double salary;
public:
    Employee(string n, int a, double s) : Person(n, a), salary(s) {}
    void display() {
        displayBasic();
        cout << "Salary: " << salary << endl;
    }
};
```

Lab Exercises

Exercise 1

Write a program that demonstrates **public inheritance**. Create a base class `Shape` with a `protected` data member `color` (string) and a `public` member function `displayColor()` that prints the color. Derive a class `Circle` from `Shape` using **public** inheritance. Add a `private` data member `radius` (double) to `Circle` and a `public` member function `area()` that computes and displays the area (πr^2, use $\pi = 3.14159$). In `main`, create a `Circle` object, set its color and radius, then call both `displayColor()` and `area()`.

Note: Clearly comment in your code which members are accessible and why.

Exercise 2

Write a program that demonstrates **protected inheritance**. Create a base class `BankAccount` with `public` member functions `setBalance(double)` and `getBalance()` and a `private` data member `balance`. Derive a class `SavingsAccount` from `BankAccount` using **protected** inheritance. Add a `public` member function `addInterest(double rate)` that increases the balance by the given percentage (using `getBalance()` inside the derived class). In `main`, create a `SavingsAccount` object and:

- Set an initial balance.
- Add 8% interest.
- Attempt to call `getBalance()` directly on the object from `main` and explain (in a comment) why this either works or fails.

Exercise 3

Write a program that demonstrates **private inheritance**. Create a base class `Logger` with a `public` member function `log(string message)` that prints "[LOG]: " followed by the message. Derive a class `FileManager` from `Logger` using **private** inheritance. `FileManager` should have a `public` member function `saveFile(string filename)` that internally calls `log()` to record the save action, then prints "File saved: " followed by the filename. In `main`, create a `FileManager` object and call `saveFile()`. Then attempt to call `log()` directly on the object and explain in a comment why this succeeds or fails.

Exercise 4

A university maintains records of **Students** and **Lecturers**, both of whom are **Persons**. You are required to implement a program using **hierarchical inheritance** as follows:

- The base class `Person` contains `protected` data members: `name` (string) and `CNIC` (string), along with a parameterised constructor and a `public` function `displayInfo()` that prints both fields.
- The derived class `Student` (public inheritance) adds `private` members `rollNumber` (string) and `GPA` (double). It has a parameterised constructor that calls the base constructor via an initialiser list, and overrides `displayInfo()` to also print the student-specific fields.
- The derived class `Lecturer` (public inheritance) adds `private` members `employeeID` (string) and `department` (string). It similarly overrides `displayInfo()`.
- In `main`, create one `Student` object and one `Lecturer` object using parameterised constructors, then call `displayInfo()` for each.

Exercise 5

A retail company organises its staff in a hierarchy: every **Employee** has an ID and a base salary; a **Manager** is an Employee who additionally supervises a department; a **RegionalDirector** is a Manager who is also responsible for a region. Implement this using **multilevel inheritance** as follows:

- `Employee` (base class): `protected` members `employeeID` (int) and `baseSalary` (double); a parameterised constructor; a `public` function `display()` that prints both fields.
- `Manager` (derived from `Employee`, public): adds `private` member `department` (string); parameterised constructor using initialiser list; overrides `display()`.
- `RegionalDirector` (derived from `Manager`, public): adds `private` member `region` (string); overrides `display()`.
- Each `display()` function must also print the bonus: 5% of `baseSalary` for Employees, 10% for Managers, and 15% for Regional Directors.
- In `main`, instantiate one object of each class using parameterised constructors and call `display()` on each.

Exercise 6

A hospital system needs to distinguish between different categories of staff. Design a program using **hierarchical inheritance** with the following class structure:

- Base class `HospitalStaff`: `protected` members `staffID` (int) and `name` (string); a virtual-like (non-virtual for this lab) `public` function `getRole()` that returns the string "Hospital Staff".
- Derived class `Doctor` (public inheritance): adds `private` member `specialisation` (string); overrides `getRole()` to return "Doctor"; adds `public` function `prescribe(string patientName)` that prints "Dr. [name] prescribed medication to [patientName]".
- Derived class `Nurse` (public inheritance): adds `private` member `ward` (string); overrides `getRole()` to return "Nurse"; adds `public` function `assist(string doctorName)` that prints "Nurse [name] assisted [doctorName]".
- Derived class `Administrator` (protected inheritance): adds `private` member `officeLocation` (string); can access `staffID` and `name` internally; has a `public` function `scheduleAppointment()` that prints a confirmation message. Comment why `getRole()` cannot be called on an `Administrator` object from `main`.
- In `main`, create objects of all three derived classes. For `Doctor` and `Nurse`, call `getRole()` and their respective specialised functions.

Exercise 7

You are required to build a small simulation of a **Vehicle Rental System** using multiple inheritance types:

- Base class `Vehicle`: `protected` data members `registrationNo` (string), `brand` (string), and `dailyRate` (double); parameterised constructor; `public` function `displayVehicle()` that prints all three fields.
- Derived class `Car` (public inheritance from `Vehicle`): adds `private` member `numDoors` (int); overrides `displayVehicle()` and also prints the number of doors.

- Derived class `ElectricCar` (public inheritance from `Car`): adds `private` member `batteryCapacity` (double, in kWh); overrides `displayVehicle()` and also prints battery capacity. Overrides a `public` function `calculateRentalCost(int days)` to apply a 15% green discount on the daily rate.
- Derived class `Truck` (private inheritance from `Vehicle`): adds `private` member `payloadCapacity` (double, in tonnes); provides a `public` function `displayTruck()` that internally calls the base `displayVehicle()` and prints payload capacity. Explain in a comment why `registrationNo` cannot be accessed directly inside `Truck` even though it is `protected` in `Vehicle` — then fix it correctly.
- In `main`, create one object of each derived class, display their information, and calculate a 7-day rental cost for the `ElectricCar`.

****